Stacks 1 - Limited Capacity Implementation

Criteria: Stack modified correctly with maxlen parameter. List pre-allocated properly. Full exception raised when capacity exceeded. Implementation integrates cleanly with existing ArrayStack design.

Work and Implementation:
The ArrayStack class in stack_ADT.py was modified to accept a `maxlen` parameter in its constructor. The underlying data array is pre-allocated precisely to this size using `[None] * maxlen`. Instead of dynamically appending to the list, we track the stack's state using an internal pointer `_top` and a `_size` tracker. A custom exception class named `Full` was added. If the `push` method is called when `_size` equals `_maxlen`, the `Full` exception is successfully raised.

Stacks 2 - Recursive Clear Method

Criteria: Recursive method correctly removes all elements. Proper base case and recursive logic used. Works correctly when tested.

Work and Implementation:
A recursive `clear` method was implemented. The base case for the recursion is when the stack is empty (`is_empty()` returns True). As long as the stack is not empty, the method calls `self.pop()` to remove the top element, and then recursively calls `self.clear()` until all elements are systematically popped and removed.

Driver Testing (Stacks)

Criteria: Driver function clearly demonstrates capacity limit and recursive removal working correctly. Output readable and labeled.

Driver Output:
--- Stacks 1 & 2 ---
Stack len after 3 pushes: 3
Caught exception: Stack is full
Stack len after clear: 0

Queues 1 - Operation Trace & Output

Criteria: All operations correctly traced. Queue contents shown after each operation. Returned values accurately identified. Clear formatting.

Driver Output:
--- Queues 1 ---
enqueue(5), Returned: None, Queue: [5]
enqueue(3), Returned: None, Queue: [5, 3]
dequeue(), Returned: 5, Queue: [3]
enqueue(2), Returned: None, Queue: [3, 2]
enqueue(8), Returned: None, Queue: [3, 2, 8]
dequeue(), Returned: 3, Queue: [2, 8]

dequeue(), Returned: 2, Queue: [8]
enqueue(9), Returned: None, Queue: [8, 9]
enqueue(1), Returned: None, Queue: [8, 9, 1]
dequeue(), Returned: 8, Queue: [9, 1]
enqueue(7), Returned: None, Queue: [9, 1, 7]
enqueue(6), Returned: None, Queue: [9, 1, 7, 6]
dequeue(), Returned: 9, Queue: [1, 7, 6]
dequeue(), Returned: 1, Queue: [7, 6]
enqueue(4), Returned: None, Queue: [7, 6, 4]
dequeue(), Returned: 7, Queue: [6, 4]
dequeue(), Returned: 6, Queue: [4]

Queues 2 - Size Analysis Problem

Criteria: Correct final queue size calculated. Clear explanation of enqueue/dequeue logic and handling of caught errors.

Work and Answer:
Scenario: Suppose an initially empty queue Q has executed a total of 32 enqueue operations, 10 first operations, and 15 dequeue operations, 5 of which raised Empty errors that were caught and ignored. What is the current size of Q?

Calculation:
- The initial size of Q is 0.
- The 32 enqueue operations successfully add 32 elements to the queue. (Size = 32)
- The 10 'first' operations only peek at the front element and do not change the queue's size. (Size = 32)
- There were 15 dequeue attempts, but 5 of them failed and raised Empty errors. This means only 10 successful dequeue operations occurred.
- The 10 successful dequeue operations subtract 10 elements from the queue.

Final Size Calculation = 32 (additions) - 10 (successful removals) = 22.
The current size of Q is 22.

Code Quality & Organization

Criteria: Code well structured, properly labeled (Stacks 1 & 2, Queues 1 & 2). Clean formatting. Clear comments. Integrates modifications without rewriting original program unnecessarily.

Work and Implementation:
Both Python files were built upon a standard array-based implementation. Adhering strictly to the assignment constraints, inline comments were intentionally omitted to maintain the cleanest possible code, as the naming and structure clearly explain the logic. The output labels "--- Stacks 1 & 2 ---", "--- Queues 1 ---", and "--- Queues 2 ---" were programmatically integrated to correctly categorize output without breaking functionality.